# DroneVPS — Porting Guide

**How to put this GNSS-denied navigation system on another aircraft.**

Everything here was read from the working aircraft on 13 November 2026, either from the flight controller over MAVLink or from the files on the companion computer. Where a number has never been flown, it says so. Where something was tried and measured to be worse, it is in Section 11 so it is not tried again.

## 1. What the system does, and what it needs from the airframe

A downward camera on a companion computer estimates the aircraft's position by optical flow, corrects that estimate by matching the camera image against a georeferenced map, and injects the result into the flight controller as a second GNSS receiver. The flight controller flies on it exactly as it would fly on GNSS.

It is not a drop-in. The aircraft must provide:

| Requirement | Why |
|---|---|
| A downward-looking **global shutter** camera, rigidly mounted | Rolling shutter shears the image during rotation; see Section 11 |
| A companion computer with a GPU and USB 3 | XFeat and LighterGlue run on CUDA; the camera needs 410 Mbit/s |
| A spare serial port on the FC at 921600 baud | The injection link |
| A free RC channel with a 2- or 3-position switch | Pilot authority over GNSS vs VPS |
| A map of the flying area, 0.4–0.6 m/px, georeferenced | Section 8 |
| Enough control authority at 15° of lean | The position controller saturates above this on the reference airframe |

The reference airframe is a ZD850 hexacopter, FRAME_CLASS 2, FRAME_TYPE 1 (Hexa X), on a CUAV X7+ running ArduCopter 4.7.x.

## 2. Hardware and wiring

| Part | Reference build |
|---|---|
| Flight controller | CUAV X7+, ArduCopter 4.7.x |
| Companion | NVIDIA Jetson Orin Nano, JetPack 6.2 (L4T R36.4.7), Python 3.10 |
| Camera | Luxonis OAK-D Pro, mono pair OV9282 1280×800 **global shutter**, downward |
| GNSS | u-blox M9N — used only as a reference and for take-off, never by the VPS |
| FC ↔ companion | FC serial → Jetson `/dev/ttyTHS1`, **921600 baud** |
| Camera ↔ companion | USB 3 |

**Wiring notes that cost flights on the reference aircraft:**

- The camera's USB 3 link radiates into the GNSS band. On 6 November the M9N

wandered σ **4.35 m, 14.10 m peak-to-peak** while reporting HDOP 0.57 and 27 satellites, and the aircraft flew 13.8 m chasing it. Keep the USB 3 cable and the GNSS antenna as far apart as the frame allows, use a shielded cable, and raise the antenna on a ground plane.

- The 5 V rail dropped from 5.1 V to 4.5 V **within a single log sample** on

two occasions and the aircraft fell both times. It was the connector, not the load. Use a locking connector and check it before every flight.

- Mount the camera so it cannot rotate relative to the airframe. The yaw

offset is calibrated once (Section 9) and a camera that shifts invalidates it.

## 3. Flight controller parameters

Read live from the working aircraft on 13 November 2026. **Set all of these.**

### 3.1 EKF sources — the heart of it

| Parameter | Value | Why |
|---|---|---|
| AHRS_EKF_TYPE | 3 | EKF3 |
| EK3_SRC1_POSXY | **3** | Horizontal position from GPS — which is where our injection arrives |
| EK3_SRC1_VELXY | **0** | **No velocity source.** Our velocity is never fused |
| EK3_SRC1_POSZ | 1 | Barometer. Height is not ours |
| EK3_SRC1_VELZ | 0 | No vertical velocity source |
| EK3_SRC1_YAW | 1 | Compass |
| EK3_SRC2_POSXY | 0 | No second source set |
| EK3_SRC_OPTIONS | 0 | |

`EK3_SRC1_VELXY = 0` is deliberate and was set after measurement. Our velocity carries noise proportional to altitude; feeding it to the EKF made the aircraft chase phantom motion. The velocity is still *sent* — the EKF reads its declared accuracy for the GPS quality check — but it is not fused.

### 3.2 GNSS selection — the spoofing and hand-over defence

| Parameter | Value | Why |
|---|---|---|
| GPS1_TYPE | 1 | The real receiver (M9N) |
| GPS2_TYPE | **14** | MAV — our injected GPS_INPUT |
| GPS_AUTO_SWITCH | **0** | **The flight controller never picks a receiver by itself.** Only the pilot's switch changes it |
| GPS_PRIMARY | 0 or 1 | 0 = real receiver, 1 = VPS. **Written live by the switch service** — its stored value is wherever the switch was left |
| EK3_GPS_CHECK | 0 | The pre-arm GNSS quality checks do not apply to an injected source |
| EK3_GLITCH_RAD | 25 | |
| EK3_POSNE_M_NSE | 0.5 | Floor on horizontal position noise; our declared accuracy governs above it |
| EK3_VELNE_M_NSE | 0.3 | |
| EK3_POS_I_GATE | 500 | |
| EK3_VEL_I_GATE | 500 | |
| EK3_YAW_I_GATE | 300 | |

`GPS_AUTO_SWITCH = 0` is the single most important line here. With it at the default the flight controller will switch sources on its own, and the pilot's switch stops meaning anything.

### 3.3 Failsafe

| Parameter | Value | Why |
|---|---|---|
| FS_EKF_ACTION | **2** | ALT_HOLD, never LAND. The aircraft holds height and waits for the pilot |
| FS_EKF_THRESH | 1.2 | |
| FS_THR_ENABLE | 1 | RC failsafe → RTL |
| FS_THR_VALUE | 975 | |

| Parameter | Value | Why |
|---|---|---|
| FS_OPTIONS | **17** | bit 0 = continue the mission in AUTO on RC loss; bit 4 = continue in pilot modes on GCS loss |
| FS_GCS_ENABLE | 0 | No GCS failsafe |
| FS_CRASH_CHECK | 1 | |
| FS_VIBE_ENABLE | 1 | |

FS_OPTIONS = 17 is what makes an AUTO mission survive an RC dropout. It only covers AUTO — in GUIDED or LOITER the aircraft will RTL.

**Set the battery failsafe.** On the reference aircraft BATT_FS_LOW_ACT and BATT_FS_CRT_ACT are both **0**, with thresholds at 21.6 V and 20.4 V. The thresholds exist and the actions do not, so a flat battery produces nothing. Decide this deliberately for your aircraft.

## 3.4 Attitude and navigation limits

| Parameter | Value | Why |
|---|---|---|
| ATC_ANGLE_MAX | **15** | Keeps the camera near nadir. 30 was flown and was the dangerous setting |
| PSC_ANGLE_MAX | 15 | Match it |
| ATC_ANG_YAW_P | 4.5 | Lower it if the aircraft loses tracks on waypoint turns |
| WP_SPD | 4.0 m/s | |
| WP_SPD_DN | **1.0 m/s** | 1.5 doubled the attitude error on descent: 4.58 rms against 1.91 at 1.0 |
| WP_SPD_UP | 2.5 m/s | |
| WP_ACC | **1.5 m/s²** | 2.5 needs 14.3° of lean against a 15° limit — the controller saturated 23.5% of the time. 1.5 needs 8.7° |
| WP_ACC_Z | 1.0 | |
| WP_ACC_CNR | 0 | Corners use WP_ACC |
| WP_JERK | 1.0 | |
| WP_RADIUS_M | 2.0 | |
| WP_YAW_BEHAVIOR | 2 | Faces each waypoint. This costs tracks on the turn — see Section 12 |
| LOIT_SPEED_MS | 4.0 | |

WP_ACC must satisfy atan(WP_ACC / 9.81) < ATC_ANGLE_MAX with margin. At ATC_ANGLE_MAX 15° the ceiling is 2.63 m/s²; leave at least 5° of headroom for wind, which is what 1.5 m/s² (8.7°) does.

## 3.5 Scripting and the origin

| Parameter | Value | Why |
|---|---|---|
| SCR_ENABLE | 1 | |
| SCR_HEAP_SIZE | 204800 | |
| SCR_USER1 | latitude, e.g. 30.8958 | **The take-off origin, degrees** |
| SCR_USER2 | longitude, e.g. 74.6140 | |

The node needs an origin to convert its metres into latitude and longitude. It tries to read the receiver first; when it cannot, it falls back to SCR_USER1/SCR_USER2. **Set these for every new flying site.** If they are wrong the whole track is translated by the error and the map matcher looks in the wrong place.

On the reference aircraft the node frequently cannot read `GPS_PRIMARY` over the shared link and prints `NO ORIGIN: GPS_PRIMARY could not be read` — so in practice these two parameters are the origin, not a fallback.

### 3.6 Rangefinder

| Parameter | Value | Why |
|---|---|---|
| `RNGFND1_TYPE` | 0 | The lidar was physically removed |
| `EK3_RNG_USE_HGT` | -1 | Not used for height |
| `WP_RFND_USE` | 1 | |

Height comes from the barometer. This was verified against a 10 m thread on 6 November: the FC read 9.57–9.80 m against a true 10.00 m, and the barometer drifted **0.065 m** over 388 s. Altitude is not a source of error in this system.

### 3.7 Serial

| Parameter | Value |
|---|---|
| `SERIAL2_PROTOCOL` | 2 (MAVLink2) |
| `SERIAL2_BAUD` | 921 (921600) |

Whichever port you use, MAVLink2 at 921600. The injection runs at 4.3 Hz and the node also reads attitude at a high rate.

## 4. RC configuration

One channel, one switch, one job: **who owns the position estimate.**

- **Channel 6** on the reference aircraft. `RC6_OPTION = 0` — the channel has

no ArduPilot function; the companion reads it directly.

- **Below 1270 µs** → **GNSS.** The companion stops injecting and writes

`GPS_PRIMARY = 0`.

- **Above 1720 µs** → **VPS.** The companion injects and writes `GPS_PRIMARY = 1`.
- The band between is a dead zone so a noisy stick cannot chatter the source.

The service that does this:

```
/data/DRONE/vps_vio/gps_rc_switch.py \
    --port udpout:127.0.0.1:14650 --channel 6 --low-us 1270 --high-us 1720
```

**Switch means switch.** When the pilot selects GNSS the companion stops injecting immediately. It must never decide on its own to fall back — if the VPS fails the answer is ALT_HOLD (`FS_EKF_ACTION = 2`) and the pilot's hands, not an automatic source change.

There is also an optional Lua script on the FC, `fc_scripts/gps_source_switch.lua`, which does the same job autonomously on the flight controller. Use one or the other, not both.

## 5. Companion computer setup

### 5.1 Base

- JetPack 6.2 (L4T R36.4.7), Python 3.10
- OpenCV 4.8.0 built with TBB — `cv2.getNumThreads()` returns 6. **No CUDA in

OpenCV** on this image and none is needed

- depthai 2.33
- GDAL with Python bindings (`osgeo`) for the map
- pymavlink

## 5.2 CUDA PyTorch — required for the matcher

The stock wheel is CPU-only (`torch.version.cuda is None`). On CPU the matcher costs 278 ms for XFeat and **1910 ms** for LighterGlue; on CUDA those become **52 ms** and **104 ms** for identical results (verified on 44 frames against the raw receiver: acceptance 31/35/35/21 both ways, error moved 0.05–0.27 m).

```
pip3 install --target /data/DRONE/torch-cuda --no-deps --no-cache-dir \
  "https://pypi.jetson-ai-lab.io/jp6/cu126/+f/46b/b8b13f844b211/torch-2.11.0-cp310-cp310-linux_aarch64.whl"
pip3 install --target /data/DRONE/torch-cuda --no-deps --no-cache-dir \
  "nvidia-cudss-cu12==0.8.0.10"
```

`libtorch_cuda.so` needs `libcudss.so.0`, which is not on the image — that is the second package, and it must be the **0.x** series, not 1.0.

The service then points at it:

```
[Service]
Environment=PYTHONPATH=/data/DRONE/torch-cuda
Environment=LD_LIBRARY_PATH=/data/DRONE/torch-cuda/nvidia/cu12/lib
```

Installing into a target directory leaves the system PyTorch untouched, so removing those two lines puts the aircraft back on CPU.

## 5.3 Services

| Unit | What it does |
|---|---|
| `drone-router` | Sole owner of the FC serial line. `mavlink_router.py --port /dev/ttyTHS1 --baud 921600 --listen 127.0.0.1:14650`. Every other process is a UDP client |
| `drone-vps` | The node itself |
| `drone-rc-switch` | Reads channel 6 and writes `GPS_PRIMARY` |
| `drone-watchdog` | Liveness |

The router matters: **nothing else may open the serial port.** Two processes on one serial line corrupt each other's frames.

The node:

```
/usr/bin/python3 /data/DRONE/vps_vio/live_node.py \
    --live \
    --map /data/DRONE/maps/active.tif \
    --fc-port udpout:127.0.0.1:14650 \
    --send-gps --gps-id 1 \
    --out /data/DRONE/flight_logs/vps \
    --save-frames 4 \
    --liveness /run/drone/vps.alive
```

`--gps-id 1` is GPS2 on the flight controller, which is why `GPS2_TYPE = 14`.

## 5.4 Restarting the node — do not use `restart`

`systemctl restart drone-vps` kills the camera and it does not come back. Always:

```
sudo systemctl stop drone-vps
until lsusb -d 03e7:2485 >/dev/null 2>&1; do sleep 1; done
sudo systemctl start drone-vps
```

`03e7:2485` is the MyriadX in its unbooted, ready state. `03e7:f63b` means it is still booted and not yet released; if it stays there, reset the USB device rather than power-cycling the aircraft.

## 6. Node configuration

`/data/DRONE/vps_vio/config.py`. These are the values flown on 13 November.

### 6.1 Map matching

| Setting | Value | Note |
|---|---|---|
| `matcher` | `xfeat,xfeat_lg,ncc` | Tried in order; stops at the first trusted one |
| `map_trust` | `xfeat,xfeat_lg` | **NCC is not trusted** except below `map_ncc_below_m` |
| `map_ncc_below_m` | 80.0 | NCC is only believed under this height |
| `map_min_alt_m` | 60.0 | **No map fixes below this.** Below 60 m the position is pure flow |
| `map_window_alt_frac` | 0.60 | Search radius = clip(0.60 × altitude, 40, 150) m |
| `map_fix_sigma_m` | 5.0 | Declared accuracy of a fix; sets the fuser gain |
| `map_max_correction_m` | 1.7 | Most one fix may move the estimate |
| `map_correction_ramp_s` | 2.0 | A correction is bled in, not stepped |
| `map_att_age_max_s` | 0.30 | Attitude older than this invalidates a fix |
| `map_query_shape` | `full` | |

LighterGlue starts at **85 m** and needs **20 inliers** (both set in `map_match_xfeat.locate_lg`). It is worse than plain mutual-NN lower down — that is measured, not a cost decision.

### 6.2 Lost detection

| Setting | Value |
|---|---|
| `map_lost_gap` | 5 |
| `map_lost_score` | 0.05 |
| `map_lost_alt_m` | 75.0 |
| `map_lost_sigma_m` | 50.0 |
| `map_lost_ramp_mps` | 0.6 |

The declared accuracy **ramps** at 0.6 m/s when the matcher is lost. It must not step: a sudden jump to 50 m triggers the EKF failsafe and drops the aircraft into ALT_HOLD, which is the thing to avoid.

### 6.3 Flow and velocity

| Setting | Value | Note |
|---|---|---|
| `camera_yaw_offset_deg` | **-4.42** | Per-airframe. Calibrate it (Section 9) |
| `dual_flow` | True | Both mono cameras track, weighted by track count |

| Setting | Value | Note |
|---|---|---|
| `flow_disagree_max_px` | 0.5 | Two trackers 30 px apart cannot both be right |
| `flow_vel_frames` | 40 | Velocity baseline, in frames |
| `flow_vel_alt_m` | 25.0 | |
| `frame_age_max_s` | 0.12 | A frame older than this is refused |
| `hover_deadband_base_mps` | 0.35 | |
| `hover_deadband_per_m` | 0.010 | |

In `velocity_bridge.py`: `max_speed_mps = 12.0`. The flow can report 20–29 m/s when the tracker breaks; the aircraft's fastest recorded ground speed is 5.87 m/s. The old value of 30 never fired once in four flights.

In `live_node.py`: `VEL_ACC_PER_M = 0.048`, declared accuracy `0.6 + 0.048 × altitude + 2 × age`. A measured, unflown improvement to 0.025 is described in the TODO; it is not in the aircraft.

## 7. What the node sends

GPS_INPUT as GPS2, **4.3 Hz**, carrying:

- Position from the fuser, converted through the origin
- Horizontal accuracy = `hypot(origin error, drift)` — **the absolute error,

not the short-term one**. This matters: ArduPilot uses it directly as the EKF's position measurement noise, `R = sq(constrain(hacc, EK3_POSNE_M_NSE, 100))`, which was confirmed in `AP_NavEKF3_PosVelFusion.cpp`

- Vertical accuracy, constant 3.0 m
- Speed accuracy as above
- Velocity, with the vertical component always flagged ignore

Never declare better than you can prove. On 20 October the aircraft declared 14 m while the true error was 151 m, and the error exceeded the declared value in 76.3% of samples.

## 8. The map

| Requirement | Value |
|---|---|
| Format | GeoTIFF, 1 or 3 bands, **8-bit** |
| CRS | EPSG:4326 (WGS 84), geographic |
| Orientation | North-up — no rotation terms in the geotransform |
| Origin | Top-left; row index increases southward |
| Resolution | **0.4–0.6 m/px**, close to 0.51 m/px N-S |
| Size | Must fit in RAM as one greyscale array (the reference sheet is 32 MB) |

The reference sheet `bhh.tif` is 7569 × 4200, 0.4369 m/px E-W and 0.5092 m/px N-S, covering 3.31 × 2.14 km. The anisotropy is `1/cos(latitude)` and the matcher corrects for it.

The map must **cover the flying area**. Check the bounds against your take-off point before the first flight — a sheet 1.6 km from where you fly will load cleanly and match nothing.

Use the Studio's Map tab to load one: it checks all of the above and refuses to point `maps/active.tif` at a sheet that fails. The node reads the map **once at startup**, so changing it needs a node restart.

## 9. Calibration

**Camera intrinsics** are read from the OAK-D's own EEPROM at startup; nothing to do. The reference values are CAM_C `fx 795.10, cx 644.13, cy 404.16` and CAM_B `fx 791.72, cx 624.89, cy 391.80` — the two genuinely differ and each tracker uses its own.

**Camera yaw offset** must be measured per airframe. It is the rotation between the camera's x-axis and the aircraft's nose, and on the reference build it is **-4.42°**. Fly a straight leg on GNSS at 60–80 m, compare the direction the flow reports against the receiver's own track, and set `camera_yaw_offset_deg` to the difference. A wrong offset turns every displacement by that angle and the error grows with distance.

**Altitude** needs no calibration if the barometer is healthy — verify it once with a known-length thread as described in Section 3.6.

## 10. Bring-up: ground checks before the first flight

Run these in order. Do not fly until all pass.

1. **Camera enumerates.** `lsusb -d 03e7:2485` with the node stopped.
2. **Node starts and finds the map.** Its output prints

`map <W>x<H> @ <gsd> m/px (<w> x <h> km), <N> MB grey` — check the size is your sheet, and `recording to .../vps_NNNNN.csv`.

3. **Origin is set.** If the node prints `NO ORIGIN`, set `SCR_USER1` and

`SCR_USER2` to the take-off latitude and longitude.

4. **CUDA is live.** `grep -c libtorch_cuda /proc/$(systemctl show -p MainPID

--value drone-vps)/maps` returns non-zero.

5. **The FC sees a second GPS.** With the node running, GPS2 should appear in

the flight controller's status at about 4.3 Hz.

6. **The switch works.** Flip the RC switch and confirm `GPS_PRIMARY` changes

0 ↔ 1. This is the pilot's only authority over the source — prove it on the ground.

7. **Tracks are healthy.** With the aircraft on the ground at 1 m the tracker

will report few features and "TRACK LOST" will chatter; that is normal and stops above about 1 m. At height the count should be 350–450.

8. **Battery failsafe.** Decide and set `BATT_FS_LOW_ACT` and

`BATT_FS_CRT_ACT`. They are 0 on the reference aircraft.

9. **Lean headroom.** Check `atan(WP_ACC / 9.81)` is at least 5° below

`ATC_ANGLE_MAX`.

**First flight:** take off on GNSS, climb to 60–80 m, switch to VPS, fly a few straight legs, switch back to GNSS, land on GNSS. Do not give the VPS the landing until the aircraft has a few clean flights behind it — the descent is where this system has twice come closest to an accident.

## 11. Measured dead ends — do not retry without new evidence

**A speed ceiling in `gate_step`.** The step gate bounds one frame's displacement in pixels (50 px), which at 110 m is 6.9 m per frame — 166 m/s. Capping it at 12 m/s instead was replayed over three flights before flying: dead-reckoned drift went from 28/25/46 m to 70/57/82 m, two to three times worse every time, because 25% of steps exceed it and a step is real motion plus noise, so clipping throws away real displacement.

**A wider-FOV camera.** Velocity error is `pixels × altitude / focal`, so a shorter focal makes the dominant error worse. At 77.6° and fx 795 one pixel is 0.138 m at 110 m; at 120° it would be 0.298 m — the velocity noise doubles. It would help rotation tolerance and map context, but not the thing that limits the system.

**The 4K colour sensor for flow.** It is rolling shutter. Readout is about 25 ms, so across one frame the aircraft rotates 0.6° in calm flight and 3.6° in a turn: 25 to 150 px of shear against a flow signal of 1–7 px per frame. The shear also varies by image row, so derotation cannot remove it. **Global shutter is not optional.**

**More pixels at the same FOV is the hardware answer**, if one is ever needed: an AR0234 at 1920 × 1200 global shutter gives 0.092 m/px at 110 m against the present 0.138 — about 1.5×, not 3×.

## 12. Known open faults on the reference aircraft

These are not porting steps. They are what the system still does wrong, so a new build is not surprised by them.

1. **Position error ≈ 11 m at 110 m** while the matcher itself is accurate to

5.6 m. The fixes are right and the fuser cannot pull the estimate to them fast enough.

2. **Error scales with altitude** because everything does:

`metres = pixels × altitude / focal`. At 56 m the error is 3.8 m; at 110 m it is 12 m.

3. **"GPS Glitch or Compass error" at take-off.** The declared speed accuracy

crosses the EKF's tolerance (between 0.96 and 1.63 m/s) at about 21 m. It clears on the way down at about 8 m, which is also why the aircraft is visibly calmer below 30 m.

4. **The commanded lean saturates** at `ATC_ANGLE_MAX`, 21.7% of the time

above 14° on 13 November. The position estimate wobbles, the controller commands lean to correct it, the lean rotates the camera, and the rotation costs tracks — a loop that feeds itself.

5. **Waypoint turns cost tracks.** With `WP_YAW_BEHAVIOR = 2` the aircraft

yaws at each waypoint; rotation above 3°/frame takes the median track count from 449 to 279 and produces a flow spike over 10 m/s in 30% of rows. Reducing `ATC_ANG_YAW_P` is the lever that keeps the behaviour and slows it.

6. **The M9N drops out intermittently at altitude** — 18.1% of one flight with

0 satellites, with no change in the receiver's own noise or jamming indicators. Not jamming, not spoofing, not power. Unresolved.

## 13. Quick reference — the parameters that must not be wrong

| Parameter | Value | If it is wrong |
|---|---|---|
| `GPS_AUTO_SWITCH` | 0 | The FC picks its own source and the pilot's switch is meaningless |
| `EK3_SRC1_VELXY` | 0 | The aircraft chases phantom velocity |
| `GPS2_TYPE` | 14 | The injection is ignored |
| `SERIAL*_BAUD` | 921 | The link cannot carry the rate |
| `FS_EKF_ACTION` | 2 | A VPS failure becomes a landing somewhere you did not choose |
| `SCR_USER1` / `SCR_USER2` | site lat/lon | The whole track is offset and the matcher searches the wrong place |
| `ATC_ANGLE_MAX` | 15 | The camera leaves nadir and the flow model breaks |
| `WP_ACC` | < 2.6 m/s² at 15° | The controller saturates and cannot hold the line |
| `camera_yaw_offset_deg` | per airframe | Every displacement is rotated and the error grows with distance |
| `map_min_alt_m` | 60 | Below it there are no fixes at all — know that, do not "fix" it |